作业 HW5* 实验报告

姓名：李欣洋 学号：2453359 日期：2025年12月19日

实验报告格式要求按照模板（使用Markdown等也请保证报告内包含模板中的要素）
#
对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考vscode配色）1
实验报告要求在文字简洁的同时将内容表示清楚
报告内不要大段贴代码，尽量控制在20页以内

1. 涉及数据结构和相关背景

题目或实验涉及数据结构的相关背景
实验整体使用到的数据结构是图，多使用邻接表和邻接矩阵的形式来存储图。
题目一:
给你一个长度为n的整数数组nums和一个长度为m的整数数组queries，返回一个长度为m的数组answer，其中answer[i]是nums中元素之和小于等于queries[i]的子序列的最大长度。
子序列是由一个数组删除某些元素（也可以不删除）但不改变元素顺序得到的一个数组。
题目二:
二叉排序树BST（二叉查找树）是一种动态查找表，基本操作集包括：创建、查找，插入，删除，查找最大值，查找最小值等。
本题实现一个维护整数集合（允许有重复关键字）的BST，并具有以下功能：1. 插入一个整数 2.删除一个整数 3.查询某个整数有多少个 4.查询最小值 5.
查询某个数字的前驱（集合中比该数字小的最大值）。
题目三:
期末考试，监考老师粗心拿错了座位表，学生已经落座，现在需要按正确的座位表给学生重新排座。假设一次交换你可以选择两个学生并让他们交换位置，给你原来错误的座位表和正确的座位表，问给学生重新排座需要最少的交换次数。
题目四:
判断一棵家族树中孩子，双亲，兄弟等关系是否判断正确。
题目五:
本题针对字符串设计哈希函数。假定有一个班级的人名名单，用汉语拼音（英文字母）表示。
要求：

1. 首先把人名转换成整数，采用函数h(key)=((...(key[0] * 37+key[1]) * 37+...)*37+key[n-2])* 37+key[n-1]，其中key[i]表示人名从左往右的第i个字母的ascii码值(i从0计数,字符串长度为n，1<=n<=100)。
2. 采取除留余数法将整数映射到长度为P的散列表中，h(key)=h(key)%M，若P不是素数，则M是大于P的最小素数，并将表长P设置成M。
3. 采用平方探测法（二次探测再散列）解决冲突。（有可能找不到插入位置，当探测次数>表长时停止探测）

题目六:
设计一个类似堆栈的数据结构，将元素推入堆栈，并从堆栈中弹出出现频率最

高的元素。

2. 实验内容

5.1 和有限的最长子序列

5.1.1 问题描述

给你一个长度为n的整数数组nums和一个长度为m的整数数组queries，返回一个长度为m的数组answer，其中answer[i]是nums中元素之和小于等于queries[i]的子序列的最大长度。

子序列是由一个数组删除某些元素（也可以不删除）但不改变元素顺序得到的一个数组。

5.1.2 基本要求

简短地描述题目要求

输入:

第一行包括两个整数n和m，分别表示数组nums和queries的长度

第二行包括n个整数，为数组nums中元素

第三行包含m个整数，为数组queries中元素

输出:

输出一行，包括m个整数，为answer中元素

5.1.3 数据结构设计

```
vector<int> nums(num_len);//存放输入的数组
vector<int> queries(query_len);//存放查询的数组
vector<int> sum(num_len+1,0);//前缀和数组
```

5.1.4功能说明（函数、类）

```
//因为题目要求的是子序列的最大长度，所以我们需要先对 nums 进行排序，然后计算前缀和
    sort(nums.begin(),nums.end());
    for(int i=1;i<=num_len;i++)
    {
        sum[i]=sum[i-1]+nums[i-1];
    }
//对于每一个查询，我们都需要遍历前缀和数组，找到最大的满足条件的前缀和的下标
    for(int i=0;i<query_len;i++)
    {
        int temp=queries[i];
        for(int j=1;j<=num_len;j++)
        {
            if(sum[j]>temp)//找到第一个大于 temp 的前缀和
            {
                answer[i]=j-1;//前一个下标就是我们要找的最大长度
                break;
            }
            else if(sum[j]==temp)//如果前缀和等于 temp
            {
                answer[i]=j;//直接返回当前下标
                break;
```

```
            }
        }
        if(sum[num_len]<=temp)//如果所有前缀和都小于等于 temp
        {
            answer[i]=num_len;//返回数组的长度
        }
    }
```

5.1.5 调试分析（遇到的问题和解决方法）

此题是一个关于查找的简单问题，在实际写代码的过程中注意维护一个前缀和数组即可。

5.1.6 总结和体会

1. 题目难点

需要理解题目的核心在于“贪心思想”与“前缀和”的结合。题目要求子序列元素之和小于等于查询值，为了使长度最大，显然应该优先选择数值最小的元素。因此，难点在于想到先对数组进行排序，再构建前缀和数组，将子序列求和问题转化为在前缀和数组中查找满足条件的上界问题。

2. 边界条件判断

需要处理查询值 queries[i] 大于所有元素之和的情况（此时答案为 n），以及查询值小于数组中最小元素的情况（此时答案为 0）。代码中通过 sum[num_len] <= temp 单独处理了全选的情况，循环中的 break 处理了常规情况。

3. 时间复杂度

总体复杂度为 O(N \log N + M x N)。其中 sort 排序耗时 O(N \log N)；构建前缀和为 O(N)；对于 M个查询，代码中采用了线性扫描前缀和数组的方式，每次查询最坏需 O(N)，总查询耗时 O(M \times N)

4. 易错点

前缀和数组 sum 的大小需要定义为 num_len + 1，且 sum[0] 应初始化为0，以便处理前 i 个元素的和。在双层循环查找时，容易搞错下标对应关系，导致答案偏大或偏小1。

5. 此题的收获

掌握了处理“最大长度”类子序列问题的通用解法：排序+前缀和。深刻理解了预处理（Pre-computation）在降低查询响应时间中的作用，同时复习了C++ STL中 vector 和 sort 的基本用法。

5.2二叉排序树

5.2.1 问题描述

二叉排序树BST（二叉查找树）是一种动态查找表，基本操作集包括：创建、查找，插入，删除，查找最大值，查找最小值等。

本题实现一个维护整数集合（允许有重复关键字）的BST，并具有以下功能：1. 插入一个整数 2.删除一个整数 3.查询某个整数有多少个 4.查询最小值 5.
查询某个数字的前驱（集合中比该数字小的最大值）。

5.2.2 基本要求

输入:

第1行一个整数n，表示操作的个数；

接下来n行，每行一个操作，第一个数字op表示操作种类：

- 若op=1，后面跟着一个整数x，表示插入数字x

- 若op=2，后面跟着一个整数x，表示删除数字x（若存在则删除，否则输出None，若有多个则只删除一个），
- 若op=3，后面跟着一个整数x，输出数字x在集合中有多少个（若x不在集合中则输出0）
- 若op=4，输出集合中的最小值（保证集合非空）
- 若op=5，后面跟着一个整数x，输出x的前驱（若不存在前驱则输出None，x不一定在集合中）

输出:
一个操作输出1行（除了插入操作没有输出）。

5.2.3 数据结构设计

```
struct TreeNode
{
    int val;
    TreeNode* left;//左孩子
    TreeNode* right;//右孩子
    TreeNode(int x):val(x),left(nullptr),right(nullptr){};
};
```

5.2.4功能说明（函数、类）

```
//以删除为例
//返回值表示是否删除成功
bool delete_tree(TreeNode* &root,int val)
{
    if(root==nullptr)
    {
        return false;
    }
    if(root->val==val)
    {
        //是叶子节点，直接删除就好
        if(root->left==nullptr && root->right==nullptr)
        {
            delete root;
            root=nullptr;
        }
        //有左孩子或者右孩子 选取孩子节点来替代即可
        else if(root->left == nullptr || root->right == nullptr)
        {
            TreeNode* temp=root;
            root=(root->left==nullptr)?root->right:root->left;
            delete temp;
            temp=nullptr;
        }
        else//有左右孩子，选择右子树的最小值来替代
        {
```

```
            TreeNode* temp=root->right;
            while(temp->left!=nullptr)
            {
                temp=temp->left;
            }
            root->val=temp->val;
            delete_tree(root->right,temp->val);
        }
        return true;
    }
    else if(val<root->val)//去左子树找
    {
        return delete_tree(root->left,val);
    }
    else//去右子树找
    {
        return delete_tree(root->right,val);
    }
}
```

5.2.5 调试分析（遇到的问题和解决方法）

二叉搜索树（BST）中最复杂的操作是节点的删除，特别是当待删除节点同时拥有左子树和右子树时。需要找到该节点的前驱（左子树最大值）或后继（右子树最小值）来替代该节点，并递归地删除替代节点，指针的操作逻辑较为繁琐。

5.2.6 总结和体会

1. 题目难点

二叉搜索树（BST）中最复杂的操作是节点的删除，特别是当待删除节点同时拥有左子树和右子树时。需要找到该节点的前驱（左子树最大值）或后继（右子树最小值）来替代该节点，并递归地删除替代节点，指针的操作逻辑较为繁琐。

2. 边界条件判断

在插入和删除时，需要判断树是否为空（root==nullptr）。在查找第k值或特定值时，需处理值不存在的情况。删除操作中，需分别处理叶子节点、单子节点和双子节点三种情况。

3. 时间复杂度

BST

的所有基本操作（插入、删除、查找）的时间复杂度取决于树的高度。在最坏情况下（树退化为链表）为 O(N)，在平均情况下（随机插入）为 O(\log N)。

4. 易错点

在函数传参时，如果需要修改指针本身（如 insert 和 delete），必须使用指针的引用 TreeNode* &root，否则无法改变父节点的指向。在delete_tree中，找到替代节点的值覆盖当前节点后，容易忘记递归删除那个替代节点。

5. 此题的收获

系统掌握了二叉搜索树的底层实现原理，深入理解了递归在树形结构操作中的核心地位。通过指针引用的使用，学会了如何优雅地在递归函数中修改树的结构，避免了维护父节点指针的冗

余代码。

5.3 换座位

5.3.1 问题描述

期末考试，监考老师粗心拿错了座位表，学生已经落座，现在需要按正确的座位表给学生重新排座。假设一次交换你可以选择两个学生并让他们交换位置，给你原来错误的座位表和正确的座位表，问给学生重新排座需要最少的交换次数。

5.3.2 基本要求

输入:

两个n*m的字符串数组，表示错误和正确的座位表old_chart和new_chart，old_chart[i][j]为原来坐在第i行第j列的学生名字

人名为仅由小写英文字母组成的字符串，长度不大于5

输出:

一个整数，表示最少交换次数

5.3.3 数据结构设计

```
std::unordered_map<std::string, int> target_position;//存储目标位置
std::vector<std::string> now_position;//存储当前的位置
```

5.3.4功能说明（函数、类）

```
int solve(std::vector<std::vector<std::string>> &old_chart,
std::vector<std::vector<std::string>> &new_chart)
    {
        // 展开 old_chart 到一维数组 记录当前位置
        for (int i = 0; i < old_chart.size(); i++)
        {
            for (int j = 0; j < old_chart[0].size(); j++)
            {
                now_position.push_back(old_chart[i][j]);
            }
        }
        int position = 0;
        // 展开 new_chart 到哈希表 记录目标位置
        for (int i = 0; i < new_chart.size(); i++)
        {
            for (int j = 0; j < new_chart[0].size(); j++)
            {
                target_position[new_chart[i][j]] = position++;
            }
        }

        int res = 0;
        for (int i = 0; i < now_position.size(); i++)
        {
```

```
            while (true)
            {
                // 当前元素在目标位置上 则跳出循环
                if (target_position[now_position[i]] == i)
                    break;
                else
                {
                    // 否则交换到目标位置上
                    std::swap(now_position[i],
now_position[target_position[now_position[i]]]);
                    res++;
                }
            }
        }
        return res;
    }
```

5.3.5 调试分析（遇到的问题和解决方法）

本题本质上是求将一个排列通过交换变为另一个排列所需的最小交换次数。难点在于将问题抽象为“图的环分解”模型：最小交换次数=节点总数-环的个数。需要构建映射关系，追踪每个元素当前位置和目标位置形成的置换环。

5.3.6 总结和体会

1. 题目难点

本题本质上是求将一个排列通过交换变为另一个排列所需的最小交换次数。难点在于将问题抽象为“图的环分解”模型：最小交换次数=节点总数-
环的个数。需要构建映射关系，追踪每个元素当前位置和目标位置形成的置换环。

2. 边界条件判断

需要处理元素已经在正确位置的情况（即自环，不需要交换）。代码通过while(true)循环不断交换当前位置错误的元素，直到该位置归位，隐式地处理了各种环的大小。

3. 时间复杂度

总体复杂度为O(N)。构建target_position哈希表需遍历一次矩阵，耗时O(N)；计算交换次数时，虽然有两层循环，但每个元素最多参与一次有效的置换环遍历，均摊复杂度为线性。

4. 易错点

在使用unordered_map建立名字到目标位置的映射时，要确保键值的唯一性。在交换逻辑中，std::swap 的两个参数索引必须正确，否则会导致死循环或逻辑错误。

5. 此题的收获

学会了利用哈希表（HashMap）快速查找目标索引，极大提升了算法效率。理解了置换群中的“循环分解”思想在解决最小交换次数类问题中的应用，提升了将实际问题（座位调整）抽象为数学模型的能力。

5.4 家族树

5.4.1 问题描述



问题描述

人类学教授对居住在孤岛上的人们及其历史感兴趣。他收集了他们的家谱进行一些人类学实验。为了实验，他需要用电脑来处理家谱。为此，他把它们翻译成文本文件。以下是表示家谱的文本文件的示例。

John
Robert
Frank
Andrew
Nancy
David

每行包含一个人的名字。第一行的名字是这个家谱中最古老的祖先。家谱只包含该祖先的后代。他们的丈夫或妻子没有在家庭树上显示。一个人的孩子比父母缩进一个空格。例如，Robert和Nancy是John的孩子，Frank和Andrew是Robert的孩子。David比Robert缩进了一个空格，但他不是Robert的孩子，而是Nancy的孩子。为了用这一方法表示一个家谱，教授将一些人从家谱中排除，使得家谱中没有人有两个父母。

为了实验，教授还收集了家属的文件，并提取了每个家谱中关于两人关系的陈述语句。以下是关于上述家庭陈述语句的一些例子。

John is the parent of Robert.
Robert is a sibling of Nancy.
David is a descendant of Robert.

对于实验，他需要判断每个陈述语句是否正确。例如，上面的前两个语句是正确的，最后一个语句是错误的。但是这个任务是十分无聊的，他想通过电脑程序来判断。

需要支持的查询有以下几种：
X is a child of Y.
X is the parent of Y.
X is a sibling of Y.
X is a descendant of Y.
X is an ancestor of Y.

注意：一个人的祖先、后代、兄弟可以是自己，但父母、孩子则不行。

5.4.2 基本要求



输入

输入包含若干组测试用例，每个测试用例由一个家谱和一个陈述句集合组成。

每个测试用例的第一行给出两个整数n,m,(0<n,m<1000)，分别表示家谱中的名字和陈述语句的数目。

接下来输入的每行少于70个字符。名字的字符串仅由不超过20个字母字符组成。在家谱的第一行中给出名字前没有前导空格，在家谱中其他的名字至少缩进一个空格，表示是第一行给出的那个名字（家族的最早祖先）的后代。若家谱中一个名字缩进k个空格，则下一行最多缩进k+1个空格。

本题假定，除了最早的祖先外，在家谱中，每个人都有其父母。在同一个家谱中同样的名字不会出现两次。家谱的每一行结束的时候无多余的空格。

再接下来每句陈述句占一行。在家谱中没有出现的名字不会出现在陈述句中。陈述句中连续的单词被一个空格分开。每句陈述句在行首和行尾没有多余的空格。

用两个0表示输入的结束。

对于20%的数据，有0<n<=20, 0<m<=40，且只有1个测试集

对于40%的数据，有0<=n<=100, 0<=m<=200，且超过一个数据集

对于100%的数据，有0<=n<=5000, 0<=m<=10000，且超过一个数据集

本题拿满分需要使用哈希表，可以使用std::map

输出

对于测试用例中的每句陈述语句，程序输出一行，给出True或False。

每个测试用例后要给出一个空行，**就算只有一个测试用例。**

5.4.3 数据结构设计

```
struct TreeNode
{
    string name;//节点名字
    TreeNode *parent;//父节点
    int depth;//节点深度
    TreeNode(string name, TreeNode *parent, int depth) : name(name),
parent(parent), depth(depth) {};
};
//构建哈希表
    map<string, TreeNode *> family_tree;
```

5.4.4功能说明（函数、类）

```
// 查找该 key 值（名字）对应的父结点指针
            TreeNode *node1 = family_tree[name1];
            TreeNode *node2 = family_tree[name2];
```

```
            if (relation == "parent")
            {
                if (node2->parent == node1)
                    cout << "True" << endl;
                else
                    cout << "False" << endl;
            }
            else if (relation == "child")
            {
                if (node1->parent == node2)
                    cout << "True" << endl;
                else
                    cout << "False" << endl;
            }
            else if (relation == "sibling")
            {
                if (node1 == node2)
                {
                    cout << "True" << endl;
                }
                else if (node1->parent != nullptr && node2->parent != nullptr &&
node1->parent == node2->parent)
                    cout << "True" << endl;
                else
                    cout << "False" << endl;
            }
.....后续对 ancestor 和 descendant 的处理省略
```

5.4.5 调试分析（遇到的问题和解决方法）

此题最大的难点在于输入数据的解析（Parsing）。需要根据字符串前的空格数量来确定树的层级关系（缩进表示父子关系），并动态构建多叉树。另一个难点是“祖先”和“后代”关系的判定，需要在树上进行向上的链式遍历。

5.4.6 总结和体会

1. 题目难点

最大的难点在于输入数据的解析（Parsing）。需要根据字符串前的空格数量来确定树的层级关系（缩进表示父子关系），并动态构建多叉树。另一个难点是“祖先”和“后代”关系的判定，需要在树上进行向上的链式遍历。

2. 边界条件判断

输入处理时需注意行末的特殊字符（如 Windows 下的 \r 或句号 .）。在查询关系时，需先判断两个人名是否存在于树中，若不存在直接输出 False。处理根节点（空格数为 0）时，其父节点指针应为 nullptr。

3. 时间复杂度

建树过程为 O(N)，其中 N 为人数。每次查询关系最坏情况下需要遍历树的高度 O(H)。由于使用了 map 存储名字到节点的映射，查找节点本身为 O(\log N)。

4. 易错点

getline 读取字符串后，容易遗留回车符导致名字匹配失败；利用 activate_tree 数组记录当前各层级的活跃节点时，必须在每读入一个新节点时更新对应层级的记录，否则父节点关系会连接错误。判断 sibling 时需注意“同一个人不是自己的兄弟姐妹”。

5. 此题的收获

锻炼了处理复杂格式化文本输入的能力，掌握了将缩进结构的文本转化为树形数据结构的方法。熟练使用了 map 结合指针的方式来管理树节点，使得可以通过名字 O(1) 或 O(\log N) 快速定位节点，而不需要从根遍历。

5.5 哈希表 2

5.5.1 问题描述

本题针对字符串设计哈希函数。假定有一个班级的人名名单，用汉语拼音（英文字母）表示。

要求：

1. 首先把人名转换成整数，采用函数 h(key)=((...(key[0] * 37+key[1]) * 37+...)*37+key[n-2])* 37+key[n-1]，其中 key[i]表示人名从左往右的第 i 个字母的 ascii 码值(i 从 0 计数,字符串长度为 n，1<=n<=100)。
2. 采取除留余数法将整数映射到长度为 P 的散列表中，h(key)=h(key)%M，若 P 不是素数，则 M 是大于 P 的最小素数，并将表长 P 设置成 M。
3. 采用平方探测法（二次探测再散列）解决冲突。（有可能找不到插入位置，当探测次数>表长时停止探测）

5.5.2 基本要求

输入:

第 1 行输入 2 个整数 N、P，分别为待插入关键字总数、散列表的长度。若 P 不是素数，则取大于 P 的最小素数作为表长。

第 2 行给出 N 个字符串，每一个字符串表示一个人名

输出:

在 1 行内输出每个字符串插入到散列表中的位置，以空格分割，若探测后始终找不到插入位置，输出一个'-'。

5.5.3 数据结构设计

```
// 初始化哈希表
    vector<string> hash_table(P, "");
```

5.5.4功能说明（函数、类）

```
for (int i = 0; i < N; i++)
    {
        string name;
        cin >> name;
```

```
        // 计算哈希值
        int key = get_key(name, P);
        bool found = false;
        int final_pos = -1;

        // 探测次数不应超过表长 P
        for (int j = 0; j <= P; j++)
        {
            int index;
            int k = (j + 1) / 2;
            long long offset = (long long)k * k;

            if (j == 0)
            {
                index = key;
            }
            else
            {
                if (j % 2 == 1)
                {
                    // 奇数次
                    index = (key + offset) % P;
                }
                else
                {
                    index = ((key - offset) % P + P) % P;
                }
            }

            if (hash_table[index] == "")
            {
                hash_table[index] = name;
                final_pos = index;
                found = true;
                break;
            }

            // 如果探测次数达到表长，停止
            if (j >= P)
                break;
        }

        if (i > 0)
            cout << " ";
```

```
        if (found)
            cout << final_pos;
        else
            cout << "-";
    }
```

5.5.5 调试分析（遇到的问题和解决方法）

此题难点在 难点在于实现双向平方探测法解决哈希冲突。具体的探测序列是 d_i = \pm 1^2, \pm 2^2, \dots，需要将其转化为代码逻辑，并正确处理取模运算中的负数问题。同时需要根据输入 M 找到下一个素数 P 作为表长。

5.5.6 总结和体会

1. 题目难点

难点在于实现双向平方探测法解决哈希冲突。具体的探测序列是 d_i = \pm 1^2, \pm 2^2, \dots，需要将其转化为代码逻辑，并正确处理取模运算中的负数问题。同时需要根据输入 M 找到下一个素数 P 作为表长。

2. 边界条件判断

计算哈希索引时，key - offset 可能为负数，必须使用 (a % P + P) % P 的公式将其转化为正索引。探测次数不应超过表长 P，若探测 P 次仍未找到空位（理论上平方探测不能保证遍历所有位置，但题目通常保证装载因子），应停止循环。

3. 时间复杂度

判断素数复杂度为 O(\sqrt{M})。插入操作在哈希表负载因子较低时，平均时间复杂度接近 O(1)。最坏情况下（冲突严重）为 O(P)。

4. 易错点

计算 k^2 时，若 k 较大可能会导致整数溢出，代码中强制转换 (long long)k * k 是必要的。在计算探测下标时，容易忘记区分奇数次探测（+k^2）和偶数次探测（-k^2）。

5. 此题的收获

深入理解了哈希表的工作原理及冲突解决机制（开放寻址法）。掌握了平方探测法的具体实现细节，以及在模运算中处理负数的技巧。了解了素数在哈希表长度选择中的重要性，以减少冲突概率。

5.6 最大频率栈

5.6.1 问题描述

设计一个类似堆栈的数据结构，将元素推入堆栈，并从堆栈中弹出出现频率最高的元素。

5.6.2 基本要求

输入:

第一行包含一个整数 n

接下来 n 行每行包含一个字符串（push 或 pop）表示一个操作，若操作为 push，则该行额外包含一个整数 val，表示压入堆栈的元素

输出:

输出包含若干行，每有一个 pop 操作对应一行，为弹出堆栈的元素

5.6.3 数据结构设计

```
    // 这个表记录频率
    map<int, int> freq;
    // 这个表记录每个频率对应的堆栈
    map<int, stack<int>> freq_stack;
    // 记录当前最大频率
    int max_freq = 0;
```

5.6.4功能说明（函数、类）

```
void push(int val)
    {
        // 这里填写你的代码
        // 更新频率表和频率栈
        freq[val]++;
        max_freq = max(max_freq, freq[val]);
        freq_stack[freq[val]].push(val);
    }

    int pop()
    {
        // 这里填写你的代码
        // 从最大频率栈中弹出元素
        int val = freq_stack[max_freq].top();
        freq_stack[max_freq].pop();
        freq[val]--;
        if (freq_stack[max_freq].empty())
        {
            max_freq--;
        }
        return val;
    }
```

5.6.5 调试分析（遇到的问题和解决方法）

需要在 O(1) 时间内实现 push 和 pop，且 pop 必须移除频率最高的元素；若有多个频率相同的最高频元素，需移除最接近栈顶的那个。难点在于设计复合数据结构：既要记录每个元素的频率，又要记录每个频率对应的元素列表（且保持插入顺序）。

5.6.6 总结和体会

1. 题目难点

需要在 O(1) 时间内实现 push 和 pop，且 pop 必须移除频率最高的元素；若有多个频率相同的最高频元素，需移除最接近栈顶的那个。难点在于设计复合数据结构：既要记录每个元素的频率，又要记录每个频率对应的元素列表（且保持插入顺序）。

2. 边界条件判断
pop 操作时，如果当前最高频率 max_freq 对应的栈变为空，需要将 max_freq 减 1，以回退到次高频率。push 时如果是第一次出现的元素，其频率从 0 变为 1。
3. 时间复杂度
push 和 pop 操作的时间复杂度均为 O(1)。这是通过牺牲空间换取时间实现的：使用了两个哈希表 freq 和 freq_stack，避免了遍历查找最大频率元素。
4. 易错点
在 pop 之后，容易忘记更新 freq 表中元素的频率（减 1）。另外，freq_stack 是一个 map<int, stack<int>>，要理解键是频率，值是该频率下的元素栈，这个嵌套结构容易混淆。
5. 此题的收获
学会了通过组合多种基础数据结构（Hash Map + Stack）来满足特殊的时间复杂度要求。理解了“空间换时间”的策略在算法设计中的应用，特别是如何通过维护额外的元数据（如 max_freq）来避免线性扫描。

3.实验总结

1. 思维的改进
通过本次实验，我学会了将具体问题抽象为数学模型，例如将座位调整转化为置换环分解 ，将子序列求和转化为排序后的前缀和查询 。同时，我建立了“空间换时间”的优化意识，意识到通过预处理（如前缀和数组）和维护辅助元数据（如频率栈中的多重哈希映射可以显著突破常规查找算法的性能瓶颈，不再局限于暴力求解。
2. 算法的学习
在算法层面，我深入掌握了树形结构的递归操作，特别是通过指针引用优雅地修改 BST 结构 4 以及根据文本缩进构建多叉家族树 5。此外，我不止停留在 STL 调用层面，更通过手写双向平方探测法解决哈希冲突 和设计复合数据结构（Map 结合 Stack），透彻理解了查找算法底层的实现原理、冲突解决机制以及如何通过组合基础结构来实现 O(1)的复杂度控制。
3. 收获
最大的收获在于提升了综合运用数据结构解决实际问题的能力，能够灵活组合 unordered_map、stack、stringstream 等容器来满足特定的逻辑需求。